DAY 4 OF 5 · CONCEPTS PRIMER

Everything You Need to Know to Do the Four Day 4 Tasks

A plain-language explanation of every vision, MLOps and feedback-loop idea the Day 4 hands-on tasks depend on — with a picture for each one. No prior computer-vision experience required.

> **The claim today rests on.** On industrial vision and machine-learning problems, the largest available gains are almost never in the model. They are in **the labels, the capture conditions, and the loop** that feeds production experience back into training. Day 1 gave you the diagnostic method; today applies it to the hardest industrial problem type and builds the machinery that keeps improvement running.

Part	Supports	Concepts covered
Part 1	Task 1 — Why does the model fail?	The six vision task types, what actually determines performance, dataset size versus quality, labelling guidelines and the ceiling, augmentation, the four axes of production variation
Part 2	Task 2 — Error analysis and hypotheses	The vision failure taxonomy, small objects, occlusion and contrast, class confusion, turning failures into testable hypotheses, threshold and cost, human review capacity
Part 3	Task 3 — The MLOps pipeline	The five questions MLOps answers, four versions bound together, experiment tracking, the evaluation gate, registry and serving, deployment strategies, monitoring signals and retraining triggers
Part 4	Task 4 — The feedback loop	Why active learning works, sampling strategies and the calibration dependency, the eight-step loop, feedback capture, continuous evaluation and the operational form of leakage
Appendix	All four tasks	Checklists you can apply directly, and a glossary

> **How to read a section.** Each concept has the same three parts: **what it is** in one or two sentences, **a picture**, and **why the task needs it**. If you are short of time, read the figures and their captions — they carry most of the content on their own.

Part 1 — Concepts for Task 1

Vision, data quality and labels

1.1 The six task types, and the one that changes the data problem

Task	Output	Use when	Labelling cost
Image classification	One label per image	The whole image is pass or fail	Lowest
Object detection	Boxes with class and confidence	You need to know what *and where*	Moderate
Segmentation	Per-pixel class	Extent or area matters — corrosion coverage, weld length	Highest
OCR	Text from an image	Serial numbers, nameplates, displays	Low if templates are stable
Gauge reading	A numeric value	Analogue instruments without digital output	Moderate
Anomaly detection	Normal or anomalous, with a score	**Defects are rare, varied, and cannot be enumerated**	**Lowest — normal images only**

The choice that saves the most effort. If your defects are rare and varied, anomaly detection needs only images of good parts. That is a fundamentally easier data problem than collecting and labelling enough examples of every defect type — and for many inspection tasks it is both the honest answer and the better one. It is the same argument as Day 2's predictive maintenance case: **when positives are rare, reformulate rather than fight the imbalance.**

Two tasks industrial teams consistently underestimate.
OCR on industrial surfaces. General OCR is trained on documents. Your text is stamped, etched, embossed or laser-marked on metal, at an angle, under directional lighting, sometimes worn. What helps: control the lighting angle, restrict the character set, validate against a known serial format, and reject rather than guess at low confidence.
Gauge and display reading. Fix the camera so the geometry is constant, calibrate the dial range once, and detect the needle rather than reading the whole image. **Watch for parallax** — a camera not perpendicular to a dial reads consistently high or low. Nothing in the model reveals it, the error is systematic rather than random so averaging does not help, and it is fixed with a mounting bracket rather than a network.

1.2 What drives model choice, and what actually determines performance

These are two different lists, and confusing them is expensive.

What drives the model choice	What actually determines performance
Inference location and latency · defect size relative to image size · available labelled data · explainability requirement · licensing	**Label consistency · capture control · resolution at the defect scale · coverage of real variation**

How to say it. "A well-controlled capture setup with a modest model beats an excellent model on inconsistent images, reliably. Benchmark leaderboard position is measured on datasets that look nothing like yours."

Raise licensing early — it belongs in this section, not in a legal review six months later. Several widely-used vision frameworks and pretrained weights carry copyleft licences whose terms can extend to network-delivered services, which may require a commercial licence for an internal enterprise deployment. Others restrict pretrained weights to non-commercial research use. **Establish the licence position for every model and dataset before it enters a production pipeline.** Treat it as a required check rather than a formality — it is genuinely important, and raising it earns credibility with legal and procurement.

1.3 Dataset size versus dataset quality

The most consequential table of the day, and the one that connects directly to Day 1's error analysis.

Figure 1 · What actually helps, by situation

	More data	Better labels	Better capture
Labels are inconsistent	none	**LARGE**	none
Defect smaller than the model can resolve	none	none	**LARGE**
A defect class has only 12 examples	**LARGE**	moderate	none
Lighting varies uncontrolled between shifts	small	none	**LARGE**
Model overfits a small clean set	moderate	small	small
Two inspectors disagree on borderline parts	none	**LARGE**	none

Four of these six rows are not solved by more data. Before anyone funds a collection programme, establish which row you are in.

Figure 1. Read down the "more data" column. Four of the six rows get nothing from it — and two of those four get a large effect from something considerably cheaper. Before anyone funds a collection programme, find out which row you are in.

1.4 Labelling guidelines — where the ceiling is set

1. **Measure agreement first.** Two experienced inspectors label the same 100 images independently. Report raw agreement and Cohen's kappa. This is your ceiling.
2. **Write the definition down.** What qualifies, what does not, and what to do at the boundary.
3. **Include visual examples of every borderline case.** A picture of the smallest mark that counts as a defect is worth pages of definition.
4. **Define an ambiguous category.** Give annotators somewhere to put genuinely unclear cases rather than forcing a guess that becomes training noise.
5. **Re-label a subset and measure again.** If agreement did not rise, iterate before labelling at scale.
6. **Version the guideline.** Labels from different guideline versions are not comparable.

Figure 2 · Label consistency is a hard ceiling on model quality

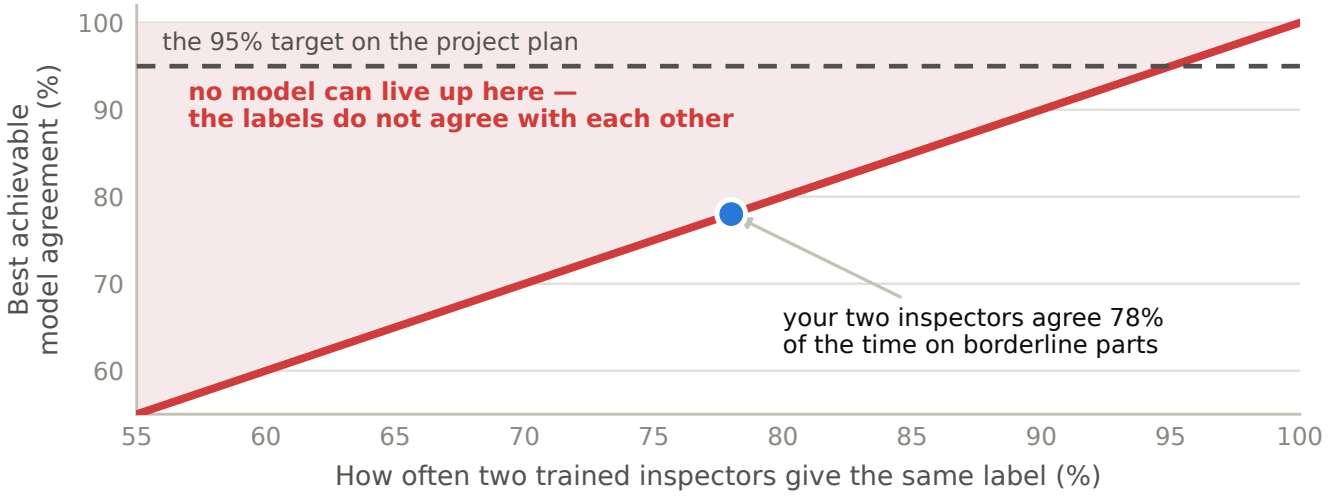


Figure 2. Your model's realistic ceiling is roughly where your inspectors agree with each other. If two inspectors agree 78% of the time, a 95% target is not ambitious — it is asking the model to be more consistent than the ground truth it learns from.

> **The inspection standard.** No quality department would run a manual inspection process without a written standard and boundary samples — every serious manufacturing operation has a limit sample cabinet. A labelling guideline is exactly that, for the training data. Teams that would never accept undefined manual inspection routinely accept undefined labelling.

1.5 What a label audit actually finds

Figure 3 · What a label audit typically finds — and none of the last three are fixed by a better model

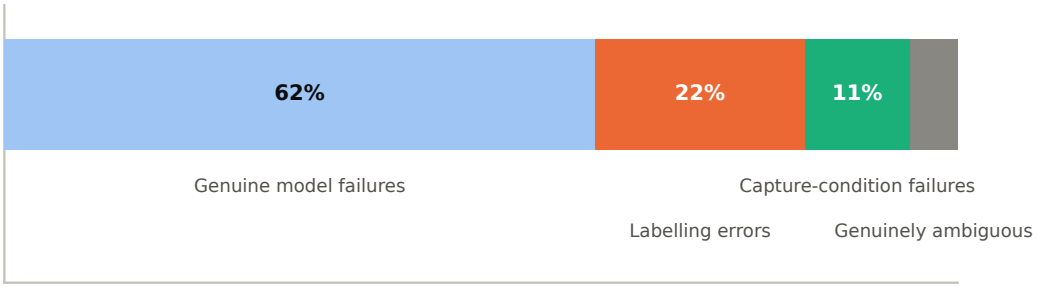


Figure 3. Of 100 cases a team reports as "the model got it wrong", a large minority are not model failures at all. Every one of the last three categories is a case where more data makes things worse or makes no difference — and none of them is visible in any metric you can compute without looking at the images.

1.6 Augmentation — every one is a claim about your process

Augmentation teaches the model what to **ignore**. That framing is what stops teams copying tutorial defaults.

Legitimate — variation that genuinely occurs	Illegitimate — a false claim about your process
Brightness and contrast within the real range	90-degree rotation with a fixed camera and fixture
Small rotations, if placement genuinely varies	Horizontal flip on an asymmetric part with fixed orientation
Noise and blur matching real sensor behaviour	Colour shifts on a monochrome setup
Scale, if working distance varies	Elastic distortion on a rigid metal component

The practical test: could this image plausibly have come out of your camera? Teaching a model to ignore orientation, when a mark on the left means something different from a mark on the right, actively destroys information it needs. Check the claim with someone who knows the line before applying it to a million images.

1.7 Production variation — four axes, none of them logged

Axis	What changes	Why it matters
Lighting	Shift changes, ambient light, ageing lamps	The most common cause of degradation after deployment
Camera and optics	Focus drift, dirt, replacement units, mounting shift	A replaced camera is a distribution shift nobody logs
Part presentation	Fixture wear, orientation tolerance, oil film	Small drifts accumulate into a new distribution
Product mix	New variants, revised parts, alternative suppliers	Invisible to monitoring until failures appear

The cheap recommendation, and it is almost never in place. Every one of these is a change to the *physical process* that nobody records as a change to the AI system — and each will show up as unexplained model degradation weeks later. **Keep a change log linking physical process changes to model monitoring.** It costs nothing.

Part 2 — Concepts for Task 2

2.1 The vision failure taxonomy

Failure mode	Signature	First hypothesis
Small object	Recall drops sharply with defect size	Increase resolution or tile before inference
Low contrast	Failures cluster on particular finishes or lighting	Improve capture lighting; add matched training data
Occlusion	Defect partly hidden	Add occluded examples; consider segmentation
Class confusion	Two classes systematically swapped	The class definitions overlap — a *labelling* problem
Poor labelling	Annotators disagree on the failures	Rewrite the guideline with boundary examples, re-label
Class imbalance	One class few examples, poor recall	Targeted collection, class weighting, or merge
Domain shift	Performance fell after a physical change	Recapture training data under current conditions
Production drift	Gradual decline, no single cause	Scheduled evaluation on a fresh labelled sample
Threshold misplacement	The precision–recall mix is wrong	Cost-optimal threshold — **no retraining needed**

> **The line to emphasise.** Two of these — **poor labelling** and **threshold misplacement** — need no new data and no retraining. They are also the two teams check last.

2.2 Small objects, and the question to ask first

A defect occupying 20 pixels in a 4,000-pixel-wide image is 0.5% of the width. Most detection architectures downsample it out of existence before the detection head ever sees it.

Approach	How it works	Cost
Higher input resolution	Feed the network a larger image	Memory and latency rise sharply
Tiling	Overlapping tiles, inference on each, merged	Linear increase in inference count
Higher optical magnification	Change the physical setup so the defect is larger	**Best result where feasible**
Multi-scale architecture	Feature pyramid designed for scale variation	Moderate; a model choice
Two-stage inspection	Fast pass locates regions; slow pass inspects	Often the best latency–accuracy trade

> **The first question to ask, before choosing any of them.** How many pixels does the smallest defect you must catch occupy? Establish that number first. **If it is under about twenty, the honest fix is optical rather than algorithmic** — and no amount of architecture or data will change it.

2.3 When the honest fix is optical

Condition	Why the model fails	What actually fixes it
Partial occlusion	The distinguishing feature is hidden	Change the fixture or add a viewpoint
Low contrast	Defect and surface differ by few grey levels	Directional or raking lighting, polarisers
Specular reflection	Highlights saturate the sensor	Polarisers, diffuse lighting, multiple exposures
Motion blur	The part moves during exposure	Shorter exposure with more light, or a strobe
Depth of field	Part of the surface is out of focus	Smaller aperture with more light, or telecentric optics

The diagnostic that costs an hour and settles the question. Can a trained inspector reliably see the defect in this image? If not, the model cannot either, and no architecture or data volume will change that. Print twenty failure images and ask an inspector.

2.4 Class confusion — and the move nobody considers

The confusion matrix shows two classes systematically swapped. The instinct is to add training data for both. What is usually happening is that **the two class definitions overlap and your annotators were also swapping them** — the model has learned the inconsistency faithfully.

The diagnostic: take 50 images the model confused and have two inspectors label them independently, without seeing the model's answer. If the inspectors also disagree, the problem is the definition.

The move nobody considers — merging the classes. If a distinction cannot be made reliably by trained humans, carrying it forward guarantees a ceiling. Merging feels like a retreat and is usually the honest simplification. Ask what decision actually depends on telling the two classes apart; frequently the answer is none, and the distinction was inherited from a taxonomy written for a different purpose.

2.5 Threshold and cost — the gain that needs nothing

Figure 4 · The threshold is an economic decision — and it is the one teams check last

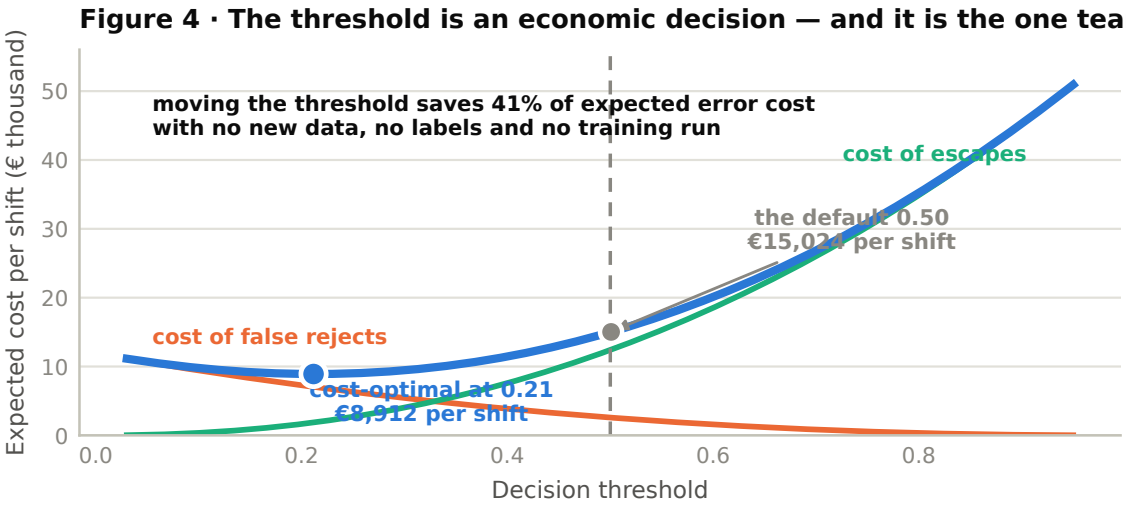


Figure 4. With a false reject costing €45 and an escape costing €1,900, the default 0.50 threshold is nowhere near optimal — and moving it requires no new data, no labels and no training run. This is the largest single available gain on many industrial systems, and it is the one teams check last.

Five steps: establish both error costs (finance owns these numbers) · sweep the threshold on held-out data and plot total cost · check per class, because the cost-optimal point frequently differs · confirm on a later time window · and compare models with **McNemar**, since both are scored on the same images and each is right or wrong per image.

An idea most teams have not considered — per-class thresholds. A safety-critical defect class set to favour recall heavily, a cosmetic class set to favour precision. One model, two operating points, no retraining.

And report per slice. An improvement that helps common classes and harms a rare critical one is not an improvement.

2.6 Human review, and the arithmetic that governs it

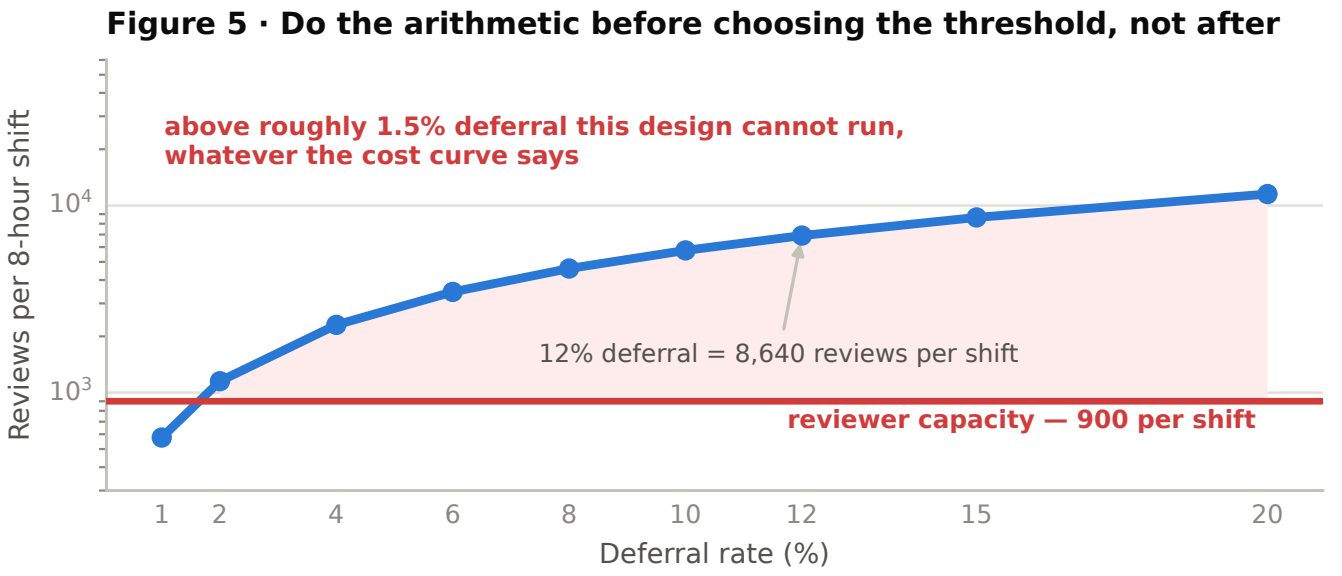


Figure 5. On a line running at two parts per second, even a modest deferral rate generates thousands of reviews per shift. The constraint here is far tighter than for document systems: review must fit inside the cycle time, or the part must be diverted to an offline station.

Six design questions: what lands in review · who reviews · how fast it must be · what the reviewer sees · what happens to the decision · and what is measured.

> **Do the arithmetic before choosing the threshold.** A review design that exceeds capacity fails on day one — and the threshold that produces it is simply not available to you, whatever the cost curve says.

Part 3 — Concepts for Task 3

3.1 What MLOps is actually for

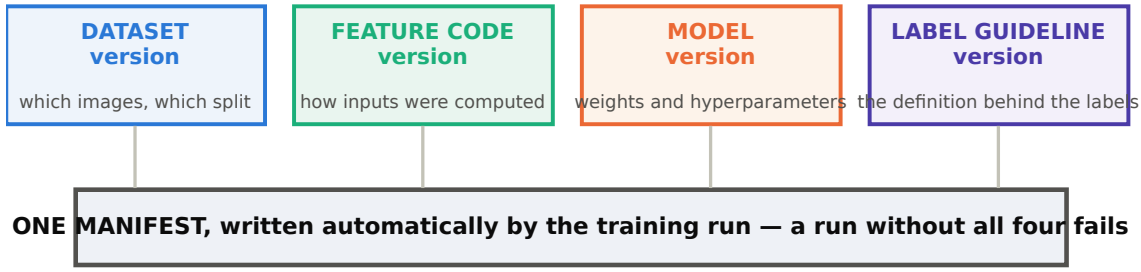
Not tooling for its own sake. Five specific questions it makes answerable:

1. Can you **reproduce** a result?
2. Can you **roll back**?
3. Can you tell it **degraded**?
4. Can you **ship a change safely**?
5. Can you **retrain reliably**?

> **How to use this list.** If the honest answer to any of the five is no, that is your MLOps priority. Do not start with the tooling — start with the unanswered question. **Buying a platform to solve a discipline problem is how organisations end up with an expensive platform and the same problem.**

3.2 Four versions, bound together

Figure 6 · Four versions, bound together — the binding matters more than the numbers



Rolling back a model without rolling back the feature code gives you a model running on inputs it was never trained for — which frequently performs worse than either version alone. Teams discover this during an incident.

Figure 6. The binding is the point. Teams version the model and consider themselves covered — and then discover during an incident that rolling the model back has left it running on feature code it was never trained against.

3.3 Experiment tracking, and the field everyone skips

Record per run: dataset and feature versions, code commit, hyperparameters, all metrics **including per-class and per-slice**, duration and cost, who ran it — and **the hypothesis it was testing**.

> **The line that gets skipped and matters most.** A tracked run without a stated hypothesis is a record that something was *tried*. A tracked run with one is a record of what was *learned* — including the negative results, which are precisely the ones teams repeat next year because nobody wrote down that they did not work.

3.4 The evaluation gate

Figure 7 · The gate is what makes it MLOps rather than continuous integration

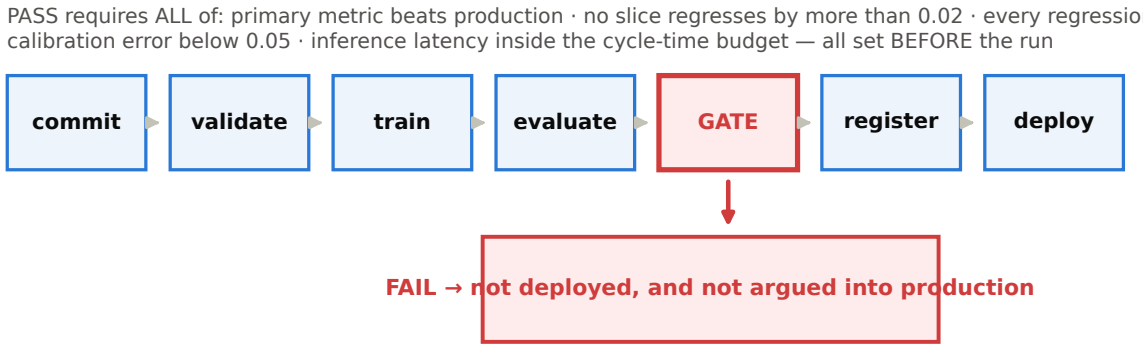


Figure 7. Seven stages, and only one of them makes this MLOps rather than ordinary continuous integration. The pass criteria are set before the run, exactly as on Day 1 — and a model that fails does not get deployed and does not get argued into production. That is what stops quality drifting downward one small exception at a time.

> **A gate that has never rejected anything is not a gate.** Push a deliberately worse model through your pipeline and watch what happens. If it deploys, you have a logging statement.

3.5 Registry, serving and deployment strategy

The **registry** records version and lineage, metrics per class and per slice, approval status and approver, which environment it serves, and known limitations. It is the prerequisite for rollback.

Strategy	How it works	Detects
Shadow	New model runs live; output not used	Real behaviour, latency, cost, disagreement rate — **at zero user risk**
Canary	Small traffic share to the new model	Real outcomes on a limited population
Blue-green	Two environments, traffic switched	Nothing new — it is a fast rollback mechanism
Staged rollout	Progressive increase with monitoring	Problems emerging only at scale or on specific segments
Champion–challenger	Challenger runs continuously alongside	Sustained comparison rather than a one-off test

> **Why shadow is unusually valuable for industrial vision.** You can compare the new model's decisions against the current one *and* against inspector judgement on the same physical parts, before anything changes on the line.
>
> **And a practice worth adopting:** once a quarter, rebuild an older model from its recorded lineage and compare. Teams that assume reproducibility without testing it usually discover, during an incident, that an undocumented step existed in someone's notebook.

3.6 Monitoring and retraining triggers

Figure 8 · Six degradation signals, ordered by how long you wait to see them

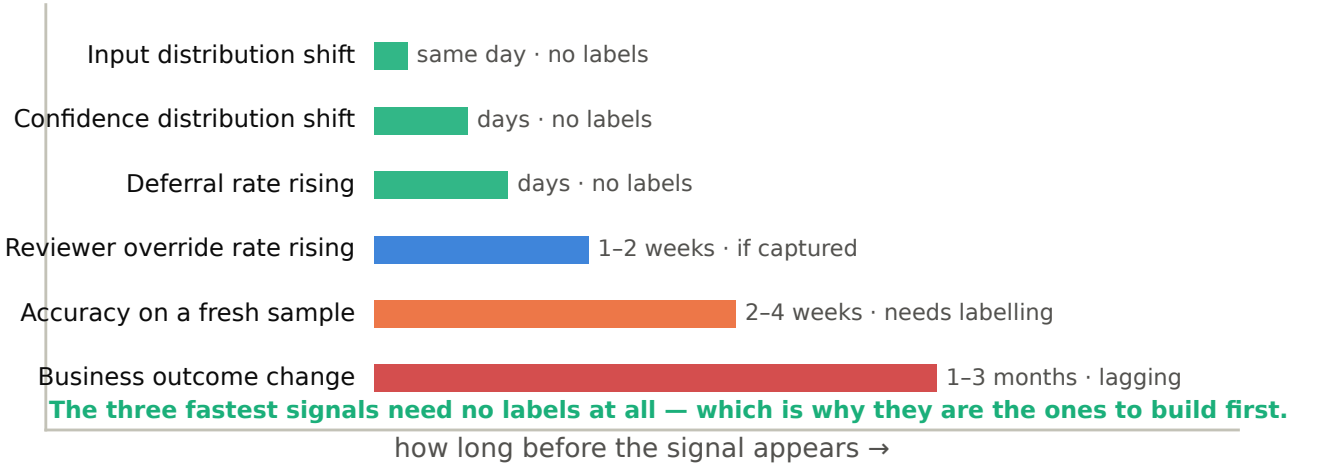


Figure 8. The signals that arrive fastest are the ones that need no labels — which is why they are the ones to build first. The signal that matters most to the business arrives last, which is exactly why you cannot rely on it alone.

Retraining triggers, set in advance: accuracy below the error budget on the fresh sample · a defined volume of new labelled data · a logged physical process change · or a scheduled cadence as a floor.

> **Triggering on a single alarming week is how you retrain on noise.** Every trigger needs a threshold *and* a duration.

Part 4 — Concepts for Task 4

4.1 Why active learning matters industrially

Random labelling: label 1,000 randomly selected images. Most are easy cases the model already handles. You have paid for 1,000 labels and bought very little information — and this is what most labelling budgets buy.

Active learning: label the 1,000 the model is least certain about, or gets wrong. Every label lands where the decision boundary is unclear.

Figure 9 · The measurement that justifies a labelling budget

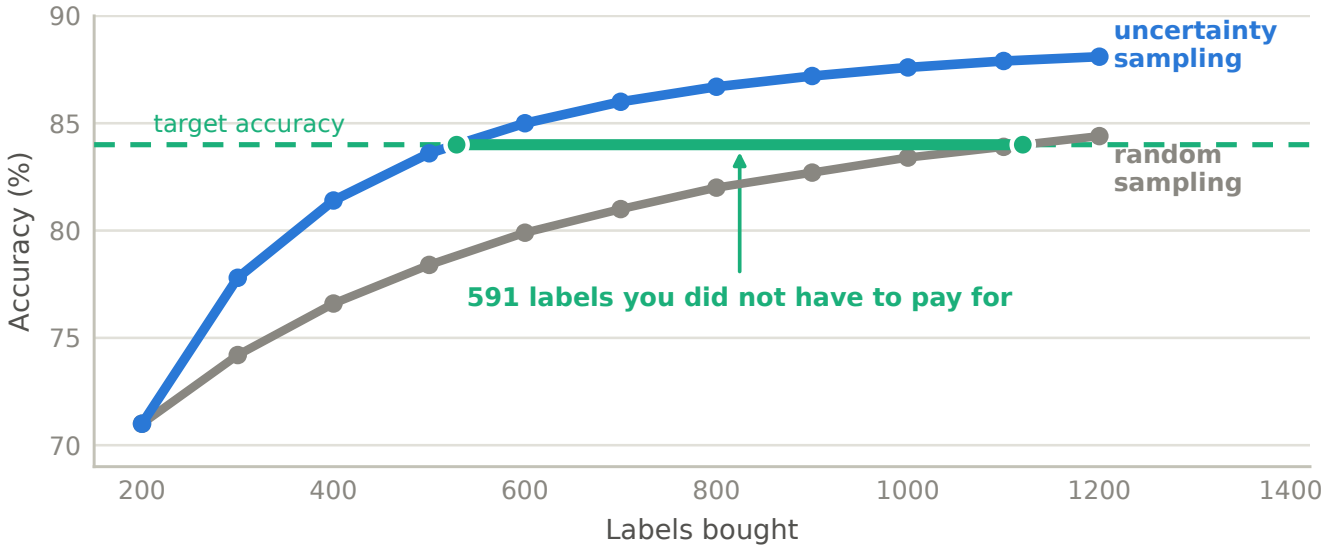


Figure 9. The horizontal distance between the curves at your target accuracy is labels you did not have to pay for. This is the measurement that justifies a labelling budget, in a form a budget holder understands immediately.

> **The apprentice.** You would not train an apprentice by having them repeat tasks they already do correctly. You would put them on the jobs they find difficult, with someone watching. Active learning is that principle applied to a training set — and it works for the same reason.

> **The link to Day 1, and the limit it implies.** Active learning targets **epistemic** uncertainty — the kind more data reduces. It will not help with **aleatoric** uncertainty. If the model is uncertain because the task is genuinely ambiguous, labelling those cases adds contradictory examples rather than information. Separating the two, as on Day 1, tells you whether the loop will pay.

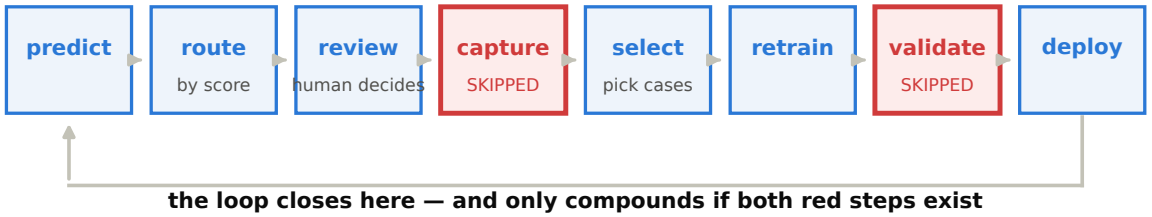
4.2 Sampling strategies, and the dependency that undermines all of them

Strategy	Selects	Strength	Weakness
Uncertainty	Predictions near the boundary	Simple, effective, cheap	**Depends on calibrated confidence**
Error-based	Cases where review overturned the model	Highest-information examples available	Needs a review loop running
Edge-case mining	Inputs unlike anything in training	Finds genuine coverage gaps	Can surface irrelevant outliers
Diversity	Spread across feature space	Avoids 200 near-identical images	More complex to implement
Disagreement	Ensemble members disagree	Directly targets epistemic uncertainty	Requires an ensemble
Class-targeted	Under-represented classes	Fixes known imbalance	Needs a target distribution

The dependency that is easily missed. Uncertainty sampling relies on calibrated confidence. If the model is over-confident — which, per Day 1, most neural networks are — **it will select the wrong examples, and the whole loop underperforms for a reason nobody diagnoses**. Calibrate before relying on it. This is why calibration error appears in the Task 3 evaluation gate.

4.3 The production learning loop, and the two steps that get skipped

Figure 10 · Eight steps, and the two that get skipped



Capture — teams build the review interface and never store the decision in a form that can reach training, so the most valuable data the system generates is discarded daily.
Validate — teams retrain on newly collected data without an evaluation gate, so a batch of mislabelled reviews degrades the model with nobody noticing.

Figure 10. Eight steps. The loop only compounds if both red steps exist — and in most systems neither does.

The question to ask your own team. "Does your review decision get stored somewhere it can reach training?" The answer is usually no, and it is usually fixable in a week.

4.4 Feedback capture and label correction

1. **Capture the decision, not just the outcome.** What the reviewer decided, what the model proposed, the confidence, and where they disagreed.
2. **Make correcting easier than overriding.** If correcting takes four clicks and overriding takes one, you will get overrides and no training data. **The interface determines the data quality.**
3. **Attribute every correction** — which reviewer, when, under which guideline version.
4. **Reconcile disagreements.** Route to a third reviewer or a defined adjudicator. Unreconciled disagreement is training noise.
5. **Sample-audit the corrections.** Corrections are labels and can be wrong.
6. **Close the loop visibly.** Tell reviewers when their corrections improved the model.

Why the last point decides whether the loop still runs in six months. Correction rates decay within weeks if reviewers cannot see that it mattered. This is a human factor and it is decisive — more so than any technical choice in the loop.

4.5 Continuous evaluation, and leakage in operational form

Six practices: keep a **protected evaluation set** · score it on a fixed schedule · refresh it from production carefully · route expert review deliberately to boundary and disputed cases · track reviewer agreement over time · and audit the corrections themselves.

The failure to avoid, and it is Day 1's leakage lesson wearing different clothes. Newly labelled production cases going into training *and* evaluation simultaneously. This quietly inflates every future measurement, and nothing about it looks wrong. **Each new case goes to one or the other, never both** — and the assignment should be made at capture time, not at training time.

Appendix A — Checklists you can apply directly

Reference

Before funding more data — which row are you in?

inconsistent labels → better labels more data makes it worse defect below resolution → better capture more data does nothing a class with 12 examples → targeted collection more data helps, if targeted uncontrolled lighting → better capture more data does little overfitting a small set → more data genuinely inspectors disagree → better labels more data does nothing

The label audit — five numbers to produce

1. raw agreement between two independent reviewers 2. Cohen's kappa on the same sample 3. share of reviewed "model failures" that were labelling errors 4. the headline metric recomputed with corrected labels 5. the gap between 3 and 4 — this is the number to lead with

Augmentation — the test

Could this image plausibly have come out of your camera? If no, the augmentation is a false claim about your process and it destroys information the model needs.

The evaluation gate — write yours in this shape

PASS requires ALL of: primary metric > current production model no slice regresses by more than a stated margin every regression test case passes calibration error below its threshold inference latency inside the cycle-time budget FAIL → not deployed, and not argued into production.

Four versions, bound on every run

dataset version · feature-code version · model version · label-guideline version

Written automatically by the training run. A run missing any of them should fail.

Monitoring — build the label-free signals first

input distribution shift same day no labels needed confidence distribution shift days no labels needed deferral rate rising days no labels needed reviewer override rate 1–2 weeks if captured accuracy on a fresh sample 2–4 weeks needs labelling business outcome 1–3 months lagging, but the one that matters

The feedback loop — the volume check that decides the design

parts per shift × deferral rate = reviews per shift compare against reviewer capacity If it does not fit, the design does not work — whatever the cost curve says.

Appendix B — Glossary

Reference

Term	Plain meaning
Active learning	Choosing which cases to label, rather than labelling at random. Targets the cases the model finds hard.
Anomaly detection	Learns what normal looks like and flags departures. Needs only images of good parts.
Blue-green	Two environments with traffic switched between them. A fast rollback mechanism, not a testing strategy.
Calibration	Whether a stated confidence matches the observed frequency. Uncertainty sampling depends on it entirely.
Canary	Routing a small share of traffic to a new model to see real outcomes on a limited population.
Capture schema	What is stored per human review. Determines whether the review can ever reach training.
Champion–challenger	A challenger model running continuously alongside production for sustained comparison.
Cohen's kappa	Agreement between two annotators, corrected for the agreement chance alone would produce.
Deferral rate	The share of cases routed to a human. Multiplied by throughput, it is a staffing number.
Domain shift	Production conditions differ systematically from training conditions.
Epistemic uncertainty	The model does not know because it has not seen enough of this kind of case. More targeted data helps.
Aleatoric uncertainty	Irreducible ambiguity in the task itself. More data does not help; more labels add contradiction.
Evaluation gate	Pass criteria set before a run that a model must meet to be deployed. What makes MLOps more than CI.
Label guideline version	The definition labels were created under. Labels from different versions are not comparable.
McNemar's test	The correct test when two models are scored right or wrong on the same items.
Model registry	What is serving, what it scored, who approved it, and its known limitations. The prerequisite for rollback.
Parallax	A camera not perpendicular to a dial reads consistently high or low. Systematic; averaging does not help.
Per-class threshold	Different operating points for different defect classes. One model, no retraining.
Segmentation	Per-pixel classification. Use when extent or area matters. Highest labelling cost.
Shadow deployment	A new model running live with its output unused. Real behaviour at zero user risk.
Tiling	Running inference on overlapping crops so small defects survive downsampling.
Training–serving skew	Features computed differently in training and production. Prevented by binding the feature version.
Uncertainty sampling	Selecting the cases nearest the decision boundary for labelling. Requires calibrated confidence.

> **If you remember five things.** Diagnose before collecting — four of six common situations are not solved by more data. Label consistency sets the ceiling. Every augmentation is a claim about your process. The evaluation gate is what makes MLOps real. And a feedback loop only compounds if capture and validation are both present.